Bilkent University
Department of Computer Engineering

Senior Design Project
T2508
Mergent

Final Report

22103002, Hasan Kutluhan Şıpka, kutluhan.sipka@ug.bilkent.edu.tr
22102787, Erdem Aydın, aerdem@ug.bilkent.edu.tr
22103227, Kaan Aydeniz, kaan.aydeniz@ug.bilkent.edu.tr
22103448, Oğuzhan Genç, oguzhan.genc@ug.bilkent.edu.tr
22102970, Furkan Emre Kolukırık, emre.kolukirik@ug.bilkent.edu.tr

Supervisor: *Eray Tüzün*
Course Instructors: *Mert Bıçakçı, İlker Burak Kurt*

13.03.2026

This report is submitted to the Department of Computer Engineering of Bilkent University in partial fulfillment of the requirements of the Senior Design Project course CS491/2.

Contents

1. Introduction **4**
1.1 Purpose of the System 4
1.2 Design Goals 4
1.2.1 Usability 4
1.2.2 Performance 4
1.2.3 Reliability 4
1.2.4 Marketability 4
1.2.5 Extendibility 5
1.2.6 Security 5
1.2.7 Scalability 5
1.2.8 Maintainability 5
1.2.9 Flexibility 5
1.3 Definitions, Acronyms, and Abbreviations 5
1.4 Overview 6
2 Current System **6**
3 Proposed Software Architecture **7**
3.1 Overview 7
3.1.1 Architectural Pillars 7
3.1.2 Runtime Pipeline: Request and Analysis 7
3.1.3 Data Strategy: Polyglot Persistence 8
3.1.4 Modular Agent Design 8
3.2 Subsystem decomposition 9
3.2.1 Issue Linkage Agent 9
3.2.2 Issue Alignment Agent 9
3.2.3 AI Code Review Agent 10
3.2.4 PR Enrichment Agent 10
3.2.5 Vibe Reviewing Agent 10
3.2.6 Post-Review Summary Agent 11
3.2.7 Data Dashboard 11
3.3 Persistent data management 11
3.3.1 Operational & Relational State (PostgreSQL) 11
3.3.2 Structural & Semantic Knowledge (Graph & Vector Stores) 12
3.3.3 Data Flow & Continuity 12
3.4 Access control and security 12
3.4.1 Automated Service Authentication 13
3.4.2 Human Administrative Control (Dashboard) 13
4 Subsystem Services **14**
4.1 Issue Linkage Agent 14
4.2 Issue Alignment Agent 16
4.3 AI Code Review Agent 18

4.4 PR Enrichment Agent 19
4.5 Vibe Reviewing Agent 21
4.6 Post-Review Summary Agent 23
4.7 Data Dashboard 24
5 Test Cases (functional and non functional test cases, procedures and expected results) 26
5.1 Issue Linkage Agent Test Cases 26
5.2 Issue Alignment Agent Test Cases 28
5.3 AI Code Review Agent Test Cases 31
5.4 PR Enrichment Agent Test Cases 34
5.5 Vibe Reviewing Agent Test Cases 36
5.6 Post-Review Summary Test Cases 38
5.7 Data Dashboard Test Cases 41
6 Consideration of Various Factors in Engineering Design 44
6.1 Constraints 45
6.2 Standards 46
7 Teamwork Details 46
7.1 Contributing and functioning effectively on the team 46
7.2 Helping creating a collaborative and inclusive environment 47
7.3 Taking lead role and sharing leadership on the team 47
8 References 48

Detailed Design Report

Mergent

1. Introduction

1.1 Purpose of the System

Code reviews are an essential part of software development, but often suffer from being slow, inconsistent, and unclear. Mergent is an AI-powered intelligent code review assistant designed to mitigate the inefficiencies of manual review cycles, which can take 15–20 hours in large-scale projects. By addressing the high volume of noisy or irrelevant feedback-which accounts for up to 36% of all review comments-the system streamlines the software development lifecycle to make it faster, clearer, and more consistent. Mergent operates through a structured six-phase pipeline: PR-Issue Linkage, Alignment Validation, AI-Generated Review Comments, PR Enrichment with Risk Insights, Vibe Reviewing, and Post-Review Feedback. By integrating Large Language Models (LLMs) into a human-in-the-loop workflow, Mergent bridges the gap between human expertise and AI efficiency, acting as a collaborative partner rather than a replacement.

1.2 Design Goals

1.2.1 Usability

- Mergent shall integrate seamlessly with version control systems such as GitHub or Bitbucket.
- AI feedback and calculated scores must be clear, actionable, and easy to understand.
- Interactions with the system, such as Vibe Reviewing, will occur directly within the pull request interface via interactive extensions.

1.2.2 Performance

- The core AI analysis of the pull request workflow must be completed within a maximum of 5 minutes.
- The Issue Agent and Alignment Agent shall respond within 2 minutes.
- AI comment generation must take no more than 2 minutes.
- The Vibe Reviewing phase, which relies on human collaboration, must respond in under 15 seconds to remain interactive.

1.2.3 Reliability

- AI suggestions and issue alignment checks must be accurate and trustworthy.
- The analysis must yield consistent results for similar pull requests to ensure predictability.
- The system shall gracefully handle external service failures (e.g., LLM provider outages) without crashing.
- The core service shall maintain a daily workflow uptime of at least 99%.

1.2.4 Marketability

- Mergent directly addresses the 8–16% of the working week developers spend on reviews and the 21.3% of comments dismissed as "Won't Fix".
- The system provides an Organization Monitoring Dashboard for team leads to evaluate productivity metrics before and after adopting the tool, directly demonstrating its value.

1.2.5 Extendibility

- Mergent employs a multi-agent architecture where specialized agents collaborate, allowing new agents to be seamlessly added to the pipeline in the future.

1.2.6 Security

- All user source code and PR data shall be encrypted in transit.
- Authentication will be handled securely via an Authentication Gateway using OAuth 2.0 or JWT-based tokens.
- User source code will strictly not be used to train general AI models without the user's explicit consent.

1.2.7 Scalability

- The system must handle at least 50 concurrent pull request analyses across different repositories without performance degradation.
- Each AI agent shall be scaled independently to maintain performance under high computational load.

1.2.8 Maintainability

- The system utilizes abstract interfaces and generic connectors so that changing the backend platform (e.g., migrating from GitHub to Bitbucket) only requires switching the implementation layer without rewriting core logic.

1.2.9 Flexibility

- Mergent integrates with existing project management tools, such as Jira, to suggest existing issues or generate new ones automatically.
- The system can be adapted and implemented specifically for client companies to align with their unique project management workflows and tracking software.

1.3 Definitions, Acronyms, and Abbreviations

- Bitbucket: A Git-based source code repository hosting service that serves as one of the target integration platforms for the project.
- Code Diff: The specific set of added, removed, or modified lines of code within a pull request.
- GitHub: A developer platform that allows developers to create, store, manage, and share their code; Mergent interacts with this via its API and specialized bots.
- Human-in-the-Loop: A design strategy where the AI system serves as a collaborative partner to human reviewers to improve feedback accuracy and tone.
- Issue: A task explanation, bug report, or feature request that describes the work required for a project.
- JIRA: A proprietary issue-tracking and project management tool used to align with existing client workflows.
- JWT (JSON Web Token): A secure token format standard used for the authentication and authorization of users within the system.
- LLM (Large Language Model): Advanced AI models trained on vast amounts of data, used by Mergent to interpret code context and generate natural language reviews.
- OAuth 2.0: A standard protocol for secure user authentication and access delegation.
- PR (Pull Request): A request made by a developer to merge code changes from one branch into another, which triggers the Mergent review pipeline.
- Repository (Repo): The project's source code is maintained through a version control system.

- Vibe Reviewing: A human-AI partnership phase where the system refines the tone and clarity of human reviewer comments.

1.4 Overview

The remainder of this Detailed Design Report formalizes the transportation of the analysis model into a comprehensive system design model. Section 2 details the proposed software architecture, subsystem decomposition, persistent data management, and access control policies. Section 3 outlines the individual subsystem services. Section 4 provides the planned functional and non-functional test cases to be executed during integration. Section 5 evaluates the consideration of various global, economic, and environmental factors in the engineering design, including constraints and standards. Finally, Section 6 details the teamwork methodologies and distribution of leadership roles utilized during the project's execution.

2 Current System

Qodo (formerly CodiumAI) focuses heavily on code generation, automated PR descriptions, and applying actionable code suggestions [1]. Qodo Merge offers strong AI review comments but lacks deeper enrichment signals, such as historical bug-prone area detection or repository-specific risk scoring. It also provides partial support for reviewer assistance (PR chat) [1]. However, it lacks true reviewer coaching and structured post-review feedback.

CodeRabbit acts as a near-autonomous AI reviewer, producing line-by-line comments, static code-graph analysis, and generating docstrings or sequence diagrams [2]. While powerful, CodeRabbit assumes issues are already linked and focuses primarily on code automation rather than collaborative review quality. It does not evaluate reviewer comments, provide enrichment beyond code-graph analysis, or offer post-review guidance.

LinearB addresses organizational bottlenecks, AI adoption metrics, and automation for PR routing [3]. It includes “Knowledge Share” to expose junior reviewers to expert reviews.[3]. However, it does not perform deep PR–issue alignment, risk-based enrichment, or post-review author support. Its reviewer-development features inspired Mergent’s Vibe Reviewing phase, yet these capabilities remain limited when applied directly to PR content.

CodeScene delivers maintainability and technical-debt–oriented insights, using hotspots, change coupling, and code health profiles [4]. Although it offers strong structural analysis, it does not assist with issue linkage, semantic alignment, reviewer coaching, or author-side summaries. It focuses on code health, not on review process quality.

CodeAnt AI combines compliance checks, secret scanning, and policy enforcement with AI-generated PR summaries [5]. However, it does not perform semantic issue alignment, risk scoring, reviewer-comment evaluation, or author-side feedback. It focuses more on organizational governance than collaborative review improvement.

GitHub Copilot offers AI reviews and inline suggestions[6]. However, its PR support is limited to summarization and basic recommendations. It does not provide enrichment signals, reviewer training, or post-review consolidation. Its strength is code-generation and autonomous patching—not improving human review quality.

3 Proposed Software Architecture

3.1 Overview

Mergent is designed as a distributed, event-driven platform designed to operationalize AI agents within the software development lifecycle. Rather than a simple chatbot, it functions as an autonomous integration platform where GitHub activities trigger a coordinated mesh of backend services. The system is cleanly stratified into three primary architectural concerns: GitHub Integration, Backend Orchestration, and Contextual Analysis.

3.1.1 Architectural Pillars

3.1.1.1 The GitHub Integration Layer (The Gateway)

Acting as the operational edge, this layer handles the high-volume ingress of webhooks such as pull request openings or issue comments, and the egress of user-facing artifacts including comments, code reviews, and labels. It is designed to be thin and stateless, primarily responsible for normalizing GitHub events and forwarding them to the backend for processing. This separation ensures that the user interface logic remains decoupled from the complex business rules and AI workflows.

3.1.1.2 The Backend Orchestration Service (The Control Plane)

The core of the platform is a central orchestration service that manages the lifecycle of specific agents. It serves as the decision-making hub, routing requests to the appropriate services. Its main responsibilities include coordinating specialized workflows for tasks such as AI Reviews, Issue Alignment, and PR Enrichment; abstracting interactions with Large Language Models such as OpenAI or Azure while managing prompt construction and context windows; and handling state management for user sessions, installation configurations, and operational metrics through a relational database such as PostgreSQL.

3.1.1.3 The Context Engine (The Knowledge Base)

Supporting the orchestrator is the Context Engine, a specialized graph-based service responsible for deep understanding of the codebase. Unlike standard tools that view code as flat text, the Context Engine ingests repositories to build a rich knowledge graph stored in Neo4j. This graph maps symbols, function definitions, call hierarchies, and dependencies, enabling AI agents to analyze code changes with awareness of the broader system architecture. It also manages vector embeddings to support semantic search across the repository.

3.1.2 Runtime Pipeline: Request and Analysis

At runtime, the architecture follows an asynchronous request-and-analysis pipeline. First, a developer action such as opening a pull request triggers a webhook. The GitHub Integration Layer captures this event, validates the signature, and delegates the payload to the Backend Orchestrator. Next, the Orchestrator determines which agent should be activated in which order, such as the AI Review Agent. Before invoking the language model, it queries the Context Engine to retrieve relevant cross-file references, definitions, and impact information related to the modified code. Then, the Orchestrator combines the pull request diff, the structural data obtained from the Context Engine, and the agent-specific prompt. This enriched context is sent to the language model for inference. Finally, the model output is parsed and returned to the GitHub Integration Layer, which publishes the results

back to GitHub in the form of structured feedback such as inline comments, summary tables, or issue links.

3.1.3 Data Strategy: Polyglot Persistence

The system adopts a polyglot persistence strategy to support different categories of data efficiently.

Operational data such as user metadata, repository settings, and authentication tokens is stored in PostgreSQL to ensure ACID compliance and referential integrity.

Code knowledge is managed through both graph and vector-based storage. Structural relationships in the codebase, such as imports, inheritance, and function calls, are stored in a graph database. Semantic representations of code chunks are stored in a vector index. This dual-storage strategy enables agents to reason about both structural connectivity and semantic meaning.

3.1.4 Modular Agent Design

A key architectural property of Mergent is that its intelligence is modular rather than monolithic. Logic is encapsulated into dedicated agents that share a common infrastructure. For example, the Issue Linkage Agent, which uses semantic search to identify related Jira or GitHub issues, operates independently from the Review Agent, which focuses on diff-based code analysis for detecting bugs or improvement opportunities. This modular design makes it possible to extend the platform with new capabilities, such as security scanning or architectural compliance checking, without destabilizing existing workflows.

3.2 Subsystem decomposition

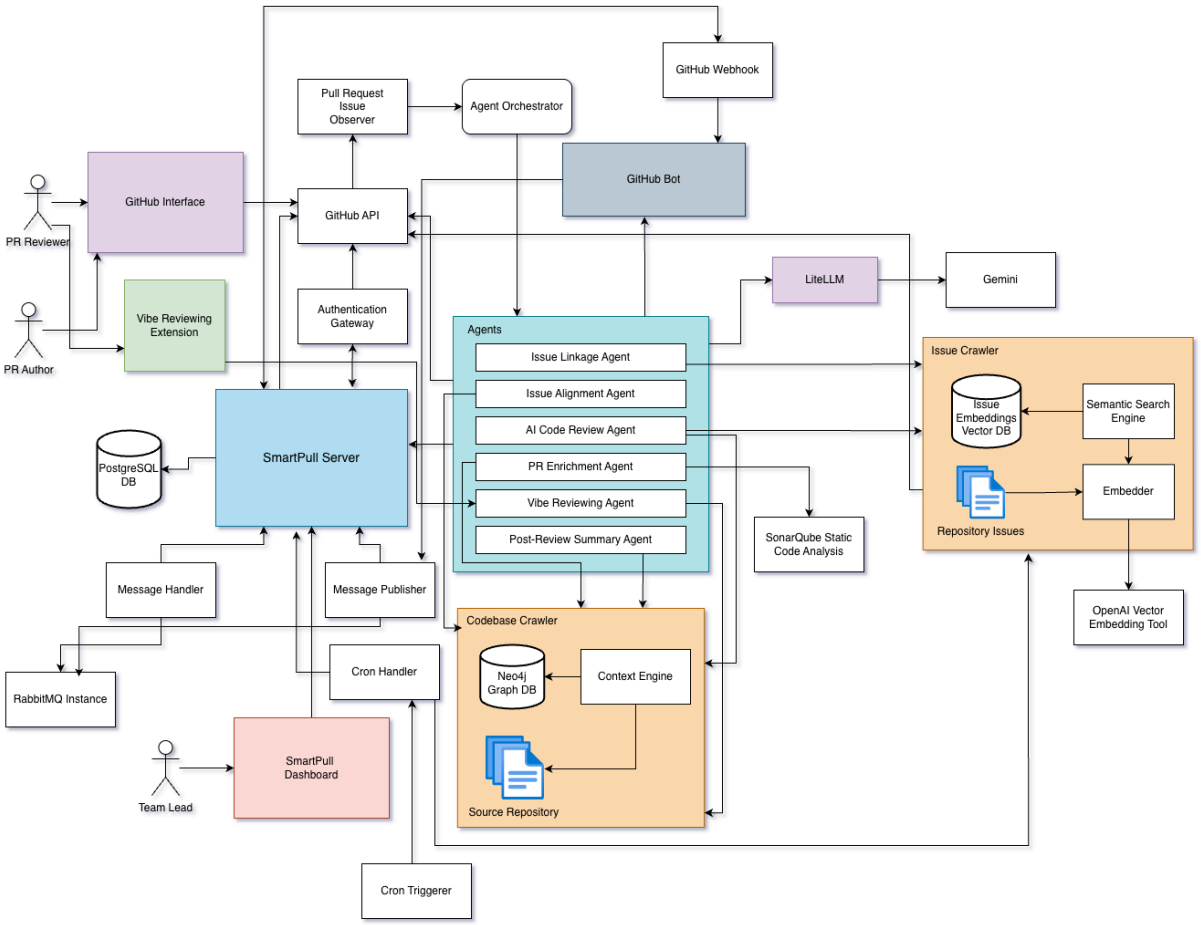

Figure 1. High Level Architecture Diagram of the System

3.2.1 Issue Linkage Agent

The Issue Linkage Agent represents the stage at which the architecture begins to associate implementation activity with tracked development tasks. Its responsibility is to analyse a newly opened pull request and determine which repository issue it is most likely intended to address. This is achieved by combining semantic similarity analysis with supporting repository-level heuristics, allowing the system to move beyond simple textual matching and consider the broader relationship between the pull request and candidate issues.

Within the software architecture, this agent serves as an early contextualisation layer. By identifying likely issue associations, it creates a foundation that can be reused by later processing stages, particularly those concerned with validation, explanation, and summarisation. In this sense, it does not merely perform a standalone retrieval task, but establishes a meaningful connection between repository events and project requirements.

3.2.2 Issue Alignment Agent

The Issue Alignment Agent is responsible for determining whether the code changes in a pull request genuinely address the expectations of the linked issue. Architecturally, it extends the work of the linkage stage by shifting the focus from issue identification to issue fulfilment. Its role is not simply to confirm that a pull request appears related to an issue, but

to evaluate whether the scope, intent, and content of the implementation are consistent with the issue's implied requirements.

This stage relies on both repository metadata and deeper contextual analysis. Through the Context Engine, the agent can examine affected code symbols, dependencies, and broader structural relationships in the codebase. Combined with model-based reasoning, this enables it to assess whether a pull request appears exact in scope, incomplete, or mixed with unrelated changes. Within the broader architecture, the Issue Alignment Agent acts as the component that links natural-language requirements to implementation-level evidence.

3.2.3 AI Code Review Agent

The AI Code Review Agent provides automated review feedback directly on the source code changes contained in a pull request. Its architectural role is to function as an intelligent first-pass reviewer capable of generating both high-level observations and line-specific review comments. Instead of depending solely on the visible diff, the agent may incorporate related file contents and broader repository context in order to produce feedback that is more accurate and relevant to the code under review.

This component occupies a central role in the architecture because it is the part of the system most directly associated with review output generation. However, its value lies not merely in automation, but in context-aware automation. By grounding its reasoning in repository structure and surrounding implementation details, it supports a form of review assistance that is more informed than conventional pattern-based comment generation. As part of the larger platform, it transforms code changes into actionable observations that can support or accelerate human review.

3.2.4 PR Enrichment Agent

The PR Enrichment Agent is responsible for expanding the architectural perspective of a pull request beyond correctness and implementation detail. Its purpose is to provide broader insight into the likely impact, risk, and significance of a change by considering factors such as affected modules, dependency propagation, and quality-related indicators. This allows reviewers to understand not only what has changed, but also where the change may have wider consequences within the repository.

From an architectural standpoint, this agent acts as a synthesis layer that combines multiple forms of analysis into an interpretable assessment. It draws on structural information from the Context Engine and may incorporate external analytical signals in order to produce a more comprehensive view of pull request significance. This helps prioritise reviewer attention and complements the more local observations generated during direct code review.

3.2.5 Vibe Reviewing Agent

The Vibe Reviewing Agent occupies a more interactive position within the architecture. Whereas earlier stages primarily analyse the pull request itself, this component is designed to support the human reviewer during the review process. Its role is to help reviewers interpret code changes, improve the clarity or tone of their feedback, and better align their comments with the broader intent of the review. As such, it introduces a collaborative dimension to the platform in which AI assists not only with technical analysis, but also with reviewer communication.

Architecturally, this agent sits closer to the user interaction layer and reflects the human-in-the-loop philosophy of the system. It leverages repository context and language model reasoning in order to assist reviewers in real time, rather than simply producing

automated outputs independently of user involvement. This makes it an important component in ensuring that Mergent enhances reviewer participation instead of attempting to replace it.

3.2.6 Post-Review Summary Agent

The Post-Review Summary Agent is responsible for consolidating the outcome of the review process once feedback has accumulated across comments, suggestions, and review exchanges. In a typical pull request, the review discussion may become fragmented across multiple locations, making it difficult for the author to extract the main concerns and required next steps. This agent addresses that problem by gathering the relevant review artefacts and transforming them into a structured summary of the review outcome.

Within the software architecture, this component serves as the closing stage of the review pipeline. Rather than focusing on source code analysis in isolation, it concentrates on synthesising the broader review interaction into a coherent result. In doing so, it improves the interpretability of the review process and reduces the effort required for developers to understand what remains unresolved. Architecturally, it functions as the layer that converts distributed review information into a more manageable and actionable form.

3.2.7 Data Dashboard

The Data Dashboard provides a longer-term analytical perspective on the operation and impact of the platform. While the other agents are primarily concerned with processing individual pull requests, the dashboard focuses on repository-level and organisational visibility. It presents historical and aggregate information related to review activity, issue linkage performance, alignment trends, contributor participation, and AI-related usage indicators. This extends the architecture beyond immediate pull request support and into continuous monitoring and evaluation.

Its role is to make the operation of the platform more observable over time. By aggregating stored workflow outputs and combining them with selected live metrics, the dashboard allows teams to examine whether review quality, efficiency, and consistency are improving. Although it does not participate directly in the execution of pull request workflows, it remains an important architectural component because it supports strategic insight and ongoing assessment of system value.

3.3 Persistent data management

Data management in Mergent acts as the backbone for both real-time workflow execution and long-term analytical intelligence. The architecture employs a polyglot persistence strategy, recognizing that operational state, structural code knowledge, and semantic meaning require distinct storage optimization models.

3.3.1 Operational & Relational State (PostgreSQL)

The primary system of record is a relational database (**PostgreSQL**) managed by the backend orchestrator. This layer handles structured entities essential for the platform's lifecycle management and multi-phase pipeline continuity:

- **Identity & Access:** Stores user sessions, organization memberships, and GitHub installation tokens to secure the platform.

- **Workflow State:** Tracks the lifecycle of repositories, pull requests, and individual analysis jobs. Because Mergent operates asynchronously, retaining state allows the system to resume complex workflows across disjointed GitHub events.

- **Historical Metadata:** Archives past review outputs, issue linkages, and alignment scores. This historical record is critical for the **Dashboard component**, enabling longitudinal analysis of code quality trends, PR velocity, and AI utilization over time.

3.3.2 Structural & Semantic Knowledge (Graph & Vector Stores)

Beyond traditional application data, Mergent persists deep structural information about the codebase itself. This is managed by the **Context Engine**, which utilizes specialized data stores to bridge the gap between raw code and AI understanding:

- **Code Property Graph (Neo4j):** The system persists a high-fidelity graph representation of ingested repositories. Nodes represent code entities (classes, functions, files), while edges represent relationships (calls, imports, inheritance). This allows agents to query the *structure* of the code—answering questions like “What functions call login()?”—without re-parsing the codebase for every request.

- **Vector Embeddings:** To support semantic retrieval, the system generates and stores high-dimensional vector embeddings of code chunks and issue text. This persistence layer enables the **Issue Linkage Agent** to perform similarity searches, finding relevant context even when keywords do not strictly match.

3.3.3 Data Flow & Continuity

This tiered persistence architecture ensures that intelligence is cumulative rather than ephemeral.

1. **Immediate Feedback:** When a PR is opened, the system retrieves the current graph state to provide instant context-aware reviews.

2. **Long-Term Learning:** As PRs are merged, the Context Engine updates the persistent graph, ensuring that subsequent reviews are based on the new state of the main branch.

3. **Analytics:** The accumulated data feeds into the reporting layer, transforming individual review outcomes into actionable insights on repository health and development productivity.

By separating operational records from structural knowledge, Mergent ensures that the system is both transactional enough for user management and sophisticated enough for deep code analysis.

3.4 Access control and security

Security in Mergent is architected around two distinct planes: automated machine-to-machine (M2M) communication for the operational pipeline, and role-based

access control (RBAC) for human administrative oversight. This dual-model approach ensures that automated workflows remain secure and scalable without compromising the granular control required by engineering leadership.

3.4.1 Automated Service Authentication

The core of the platform operates autonomously, handling events triggered by GitHub. To secure these interactions, Mergent employs a strict Machine-to-Machine (M2M) authentication strategy:

- GitHub App Integration: The primary interface for repository access is the GitHub App. Rather than relying on individual user credentials, the system authenticates using short-lived Installation Access Tokens. These tokens are generated dynamically for each specific installation (organization or repository) and are scoped to the minimum necessary permissions (e.g., read:contents, write:pull_requests).
- Service-to-Service Communication: Internal communication between the Integration Layer, Backend Orchestrator, and Context Engine is secured via internal M2M tokens. This ensures that only authorized services can trigger complex analysis jobs or access the underlying knowledge graph.
- Cron Job Security: Scheduled tasks—such as re-indexing repositories or cleaning up stale data—are protected by a dedicated Cron Token. This token authenticates the job runner against the backend APIs, preventing unauthorized external triggers of resource-intensive maintenance tasks.

3.4.2 Human Administrative Control (Dashboard)

While the review pipeline is automated, configuration and monitoring are managed through a secure Dashboard. Access to this interface is governed by a Role-Based Access Control (RBAC) system:

- Team Lead Role: The system designates a specialized Team Lead role for engineering managers or senior developers. This role grants elevated privileges to:
- Monitor Repository Health: Access the aggregated dashboard to view trends in code quality, review velocity, and issue alignment.
- Manage Installations: Add or remove repositories from the Mergent integration and manage billing or usage limits.
- Standard Member Role: Developers and contributors have read-only access to their own review history and personal settings, ensuring transparency without the risk of unintentional configuration drift.

By separating the high-frequency, automated authentication flow from the sensitive administrative control plane, Mergent minimizes the attack surface while empowering teams to manage their AI integration securely.

4 Subsystem Services

4.1 Issue Linkage Agent

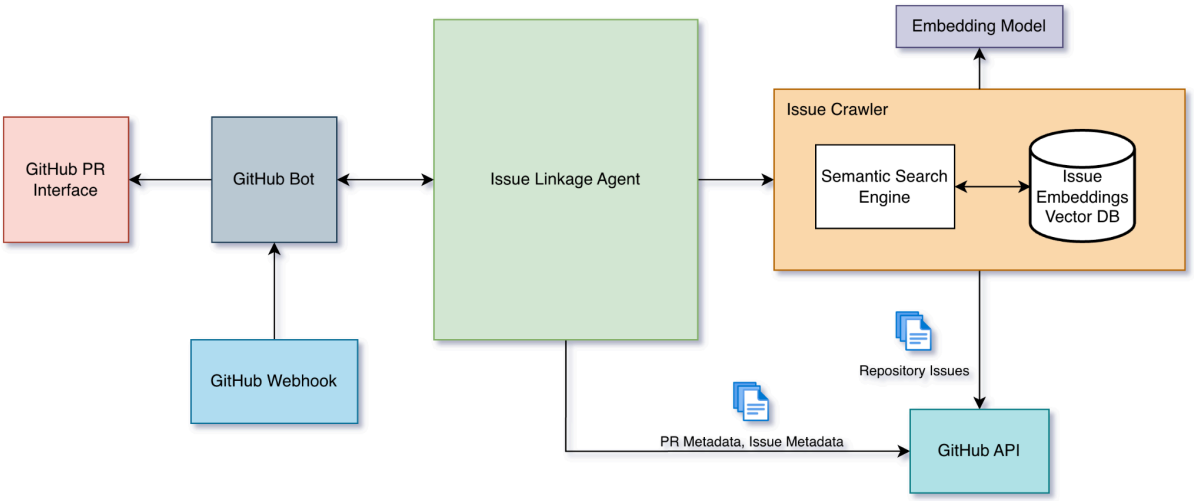

Figure 2. High Level Diagram of Issue Linkage Agent

The Issue Linkage subsystem is responsible for automatically identifying and connecting newly opened Pull Requests (PRs) to their corresponding repository issues. It bridges the gap between the GitHub platform and Mergent's semantic search capabilities.

Main Components

- GitHub Integration Layer (Webhook, Bot, & PR Interface): Acts as the communication bridge between the repository and the Mergent backend. The Webhook listens for repository events, the GitHub Bot handles incoming event payloads and outgoing actions, and the GitHub PR Interface is where the final outputs (comments and links) are displayed to the user.
- Issue Linkage Agent: The central orchestrator of this subsystem. It manages the business logic, combining semantic search results with heuristic checks (e.g., keyword matching, author history) to calculate a final confidence score for each issue.
- Issue Crawler & Semantic Search Engine: The intelligence core for finding related issues. It utilizes an Embedding Model to convert issue and PR texts (titles, descriptions) into mathematical vectors. These vectors are stored and queried within the Issue Embeddings Vector DB to find the most semantically similar matches.
- GitHub API: Used by the Agent and Crawler to fetch raw, up-to-date Repository Issues, PR Metadata, and Issue Metadata required for the analysis.

System Workflow

1. Event Trigger: When a developer opens a new Pull Request, a GitHub Webhook is triggered, which immediately notifies the GitHub Bot.
2. Data Extraction: The Bot passes the event to the Issue Linkage Agent, which then queries the GitHub API to retrieve the necessary PR metadata and the current list of repository issues.
3. Semantic & Heuristic Analysis: The Agent sends the data to the Issue Crawler. The Semantic Search Engine compares the PR's vector embeddings against the Issue Embeddings Vector DB to find semantically similar issues. The Agent then combines

these semantic similarity scores with its own heuristic checks to rank the candidate issues.

4. Execution & Feedback: Once the issues are ranked, the Agent sends the results back to the GitHub Bot. The Bot interacts with the GitHub PR Interface to post a comment listing the top 5 issue candidates. Finally, it automatically links the PR to the single issue that achieved the highest combined score.

Functions

- **generateCollectionName:** Produces a unique, deterministic name for the vector-store collection belonging to a specific repository.
- **ensureCollection:** Checks whether the vector-store collection for a repository exists and creates it with the correct dimensions if it does not.
- **generateEmbeddings:** Converts a single issue's title and body into a numeric float vector by calling the embedding model.
- **generateBatchEmbeddings:** Converts a list of issues into float vectors in a single API call, used during bulk repository indexing to reduce round-trips.
- **saveIssueEmbeddings:** Stores one issue (its vector plus all metadata) into the vector database. Used when a new issue is created or its text is edited.
- **saveIssueEmbeddingsBatch:** Stores multiple issues in a single bulk write. Used during full repository indexing to maximize throughput.
- **updateIssueMetadata:** Updates only the non-text fields of an issue record (state, closed date, assignees) without re-computing the embedding. Called on close, reopen, assign, and unassign events.
- **getAllIssues:** Retrieves every issue stored for a repository by running a vector search with a limit equal to the total record count. Returns all issues with their similarity scores.
- **getSimilarIssues:** Returns the top-K most similar issues to a query vector. Used when only the closest matches are needed rather than the full set.
- **deleteIssueEmbedding:** Removes a single issue from the vector database. Called when a GitHub issue is permanently deleted.
- **deleteCollection:** Removes the entire vector-store collection for a repository. Called when the GitHub App installation is uninstalled from that repository.
- **closeMissingIssues:** During periodic sync jobs, detects issues that are marked open in the vector database but are no longer open on GitHub, and updates their state to closed.
- **getLinkedIssues:** Calls the GitHub GraphQL API to check whether a PR already has officially linked issues declared via "Closes #N" syntax. Used as a fast path before falling back to semantic search.
- **calculateDateScore:** Scores a PR–issue pair based on how much their active date ranges overlap. Issues that were open at the same time as the PR receive a positive score; non-overlapping pairs are penalized.
- **calculateBranchScore:** Returns a fixed bonus score if the PR branch name contains the issue number as a whole word, indicating the developer intentionally named the branch after the issue.
- **calculateAssigneeScore:** Returns a fixed bonus score if the PR author is one of the issue's assignees, indicating the assigned developer submitted the fix.
- **calculateHeuristicScore:** Combines the date, branch, and assignee scores into a single breakdown object. This breakdown is added to the semantic similarity score before normalization.
- **normalizeScore:** Maps the combined raw score to a 0–100 percentage that is displayed in the linkage comment posted on the PR.
- **handleIssueLinkage:** The top-level event handler on the GitHub bot side. Drives the entire pipeline: requests scored issues from the server, formats the markdown

comment, updates the PR body with "Closes #N", posts the comment, and returns the top issue number for downstream agents.

4.2 Issue Alignment Agent

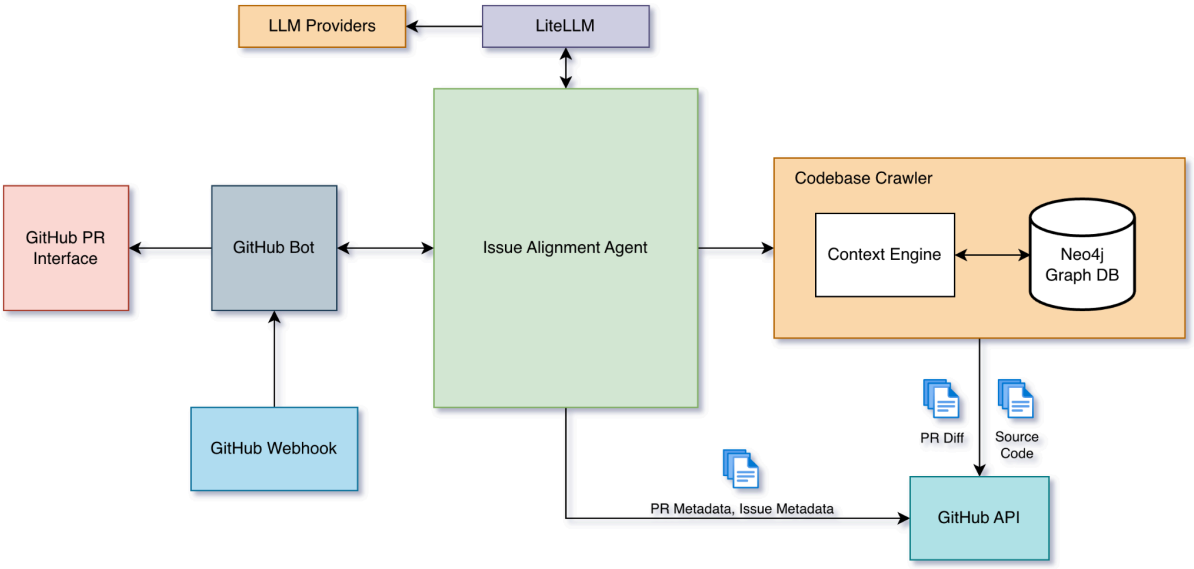

Figure 3. High Level Diagram of Issue Alignment Agent

The Issue Alignment subsystem is responsible for verifying whether the code changes in a Pull Request accurately address the requirements of its linked issue. It evaluates the code difference and structural impact to categorize the alignment state.

Main Components

- GitHub Integration Layer (Webhook, Bot, & PR Interface): Handles the standard event triggering (PR opened) and executes the final output actions, which include posting the reasoning comment and applying the appropriate alignment label to the PR.
- Issue Alignment Agent: The core orchestrator that manages the alignment workflow. It coordinates data collection, interacts with the LLM through tool calls, and processes the final categorization.
- LiteLLM & LLM Providers: The reasoning engine of the subsystem. LiteLLM acts as an intermediary proxy layer to route requests to various LLM Providers while observing, tracking, and managing token usage and performance.
- Codebase Crawler (Context Engine & Neo4j Graph DB): A static analysis engine that parses the repository to generate structural intelligence. It uses the Neo4j Graph DB to map relationships and output function call and dependency graphs for the modified code.
- GitHub API: Used by the Agent to fetch initial data (PR Diff, PR/Issue Metadata) and to dynamically retrieve specific surrounding source code chunks from the codebase when requested.

System Workflow

1. Trigger & Initial Fetch: When a PR is opened or updated, a GitHub Webhook alerts the GitHub Bot, which wakes the Issue Alignment Agent. The Agent uses the GitHub API to fetch the PR Diff, PR Metadata, and the linked Issue Metadata.
2. Structural Context Generation: Before engaging the LLM, the Agent queries the Codebase Crawler (Context Engine) to generate a function call and dependency graph associated with the files modified in the PR Diff.
3. LLM Reasoning & Tool Calling: The Agent sends the initial data (Diff, Metadata, and Dependency Graph) through LiteLLM to the LLM. By analyzing the graph and the diff, the LLM decides if it needs more surrounding context. If so, the LLM executes a tool call, instructing the Agent to fetch specific, unmodified source code chunks from the repository via the GitHub API.
4. Categorization: Once all necessary context is gathered, the LLM evaluates the scope of the code changes against the issue description and classifies the alignment into one of four categories: Exact, Missing, Tangling, or Missing & Tangling.
5. Execution & Feedback: The LLM returns its decision and detailed reasoning to the Agent. The Agent then instructs the GitHub Bot to post the reasoning as a comment on the PR Interface and formally adds a GitHub label corresponding to the identified category.

Functions

- **getFormattedAlignment:** Entry point for the alignment pipeline. Fetches PR and issue data from GitHub, initializes the LangGraph state, runs the full graph, and returns the final alignment label with its explanation.
- **buildAlignmentGraph:** Assembles and compiles the LangGraph workflow with all nodes and edges into a runnable graph instance.
- **extractRequirements:** LangGraph node. Asks the LLM to parse the issue text into a structured list of discrete acceptance criteria, each with an ID, description, and category.
- **initAnalysis:** LangGraph node. Groups the PR's changed code symbols by file path so they can be processed one file at a time by the analysis node.
- **analyzeFileSymbols:** LangGraph node. Runs a ReAct sub-agent for each file group. The agent uses knowledge-graph tools to trace each changed symbol — understanding what it does, who calls it, and what it affects — then maps it to the issue requirements or flags it as an unrelated change.
- **maybeSummarize:** LangGraph node. Compresses the accumulated analysis text when it grows too large to fit in the LLM context window, preventing failures on large PRs.
- **classifyAlignment:** LangGraph node. Final LLM call. Reads all symbol analyses and produces the alignment label and explanation that will be posted as a PR comment and applied as a label.
- **createAlignmentTools:** Factory that creates five LangChain tools for the ReAct agent, each calling the code knowledge graph backend: explore a symbol's definition and relationships, run an impact analysis, search code semantically, read a file's contents, and execute a raw Cypher query on Neo4j.
- **evaluateIssueAlignmentRequest:** Bot-side HTTP client that sends the PR and issue details to the server and returns the alignment result.
- **handleIssueAlignment:** Top-level GitHub event handler on the bot side. Extracts the linked issue number, calls the server, posts the explanation as a comment, and applies the alignment label to the PR.

4.3 AI Code Review Agent

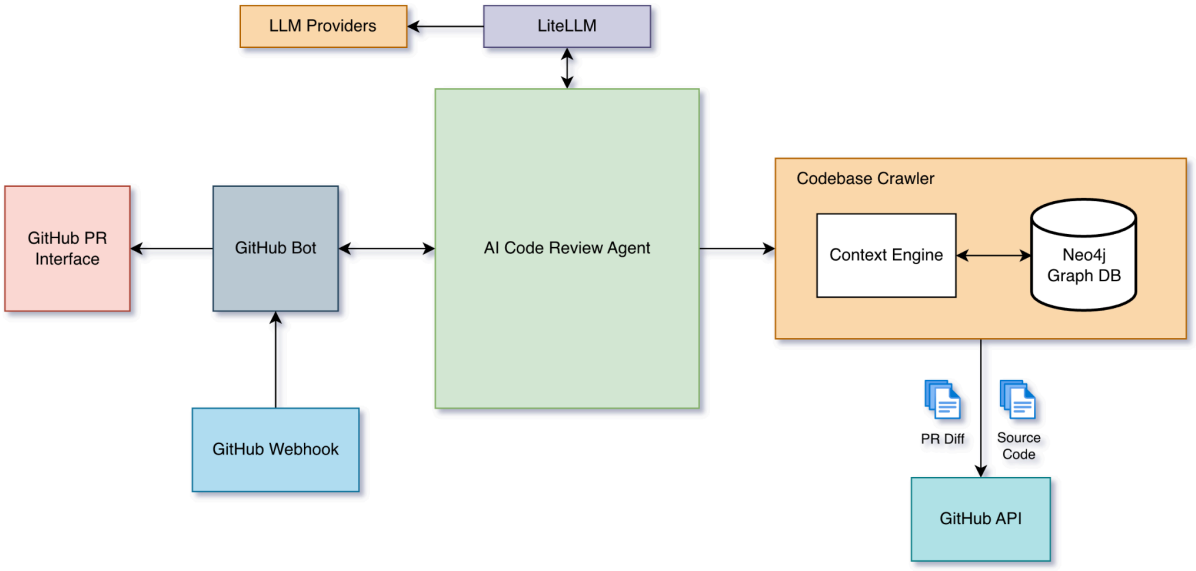

Figure 4. High Level Diagram of AI Code Review Agent

The AI Code Review subsystem is responsible for analyzing the actual code changes within a Pull Request to provide automated, actionable feedback. It acts as a first-pass reviewer by generating specific inline comments and a high-level summary of the PR.

Main Components

- AI Code Review Agent: The central orchestrator for this subsystem. It manages the flow of code data, coordinates with the LLM for analysis, and formats the output into platform-compatible review comments.
- LLM & LLM Providers: The reasoning engine that evaluates the code changes for bugs, optimizations, and best practices to generate the actual review text.
- Codebase Crawler (Context Engine & Neo4j Graph DB): Provides architectural awareness. Instead of reviewing the PR diff in isolation, this component allows the agent to understand how the modified code interacts with the rest of the system.
- GitHub Integration Layer & GitHub API: Functions similarly to previous subsystems, but here it is specifically utilized to fetch the PR Diff/Source Code and to publish formatted line-by-line comments on the PR interface.

System Workflow

1. Trigger & Data Retrieval: Initiated by a PR creation or update via the GitHub Webhook, the AI Code Review Agent is activated. It uses the GitHub API to fetch the raw PR Diff and the relevant Source Code files.
2. Contextual Enrichment: The Agent consults the Codebase Crawler to understand the structural context of the changed files (e.g., dependencies, function call graphs) mapped within the Neo4j Graph DB.
3. LLM Analysis & Tool Calling: The PR Diff and structural context are sent to the LLM. To ensure the review is accurate and lacks hallucinations, the LLM can utilize tool calls to request additional, unmodified surrounding code chunks from the codebase if it needs deeper context to understand a specific change.
4. Feedback Generation: Based on the comprehensive code context, the LLM generates specific, actionable line-by-line review comments targeting exact lines in the diff, alongside a concise overall PR summary.

5. Execution: The Agent processes the LLM's output and instructs the GitHub Bot to post the overall summary as a general PR comment and map the specific line-by-line feedback directly onto the code diff in the GitHub PR Interface.

Functions

- **generateCodeSummary:** Fetches the PR metadata, the unified diff, and the list of changed files from GitHub, then asks the LLM to produce a markdown summary of the PR's purpose and changes.
- **publishFileLevelCodeReviews:** Fetches the PR diff and each changed file's full content, sends everything to the LLM in one call, receives structured per-line comments, converts each comment's file line number to a diff position, and posts them as an official GitHub PR review.
- **generatePrSummary:** Formats the PR data, diff, and file list into the pr_summary prompt and invokes the LLM client, returning the raw summary text.
- **generateReviewComments:** Formats all changed files' content and diffs into the code_review prompt and invokes the LLM. Parses the JSON-structured response into a list of review comment objects (file path, line number, comment body, diff side).
- **calculatePositionFromDiff:** Walks a unified diff string hunk by hunk and converts an absolute file line number to the diff position integer required by the GitHub review API. Returns null if the line does not appear in the diff.
- **generateAIReviewRequest:** Bot-side HTTP client that requests the PR summary from the server and returns the resulting markdown string.
- **publishFileLevelCodeReviewsRequest:** Bot-side HTTP client that triggers the server to generate and publish all inline review comments for the PR's changed files.
- **handleAIReview:** Top-level GitHub event handler on the bot side. Calls the summary endpoint and posts the comment, then calls the inline review endpoint to publish line-level feedback on the PR.

4.4 PR Enrichment Agent

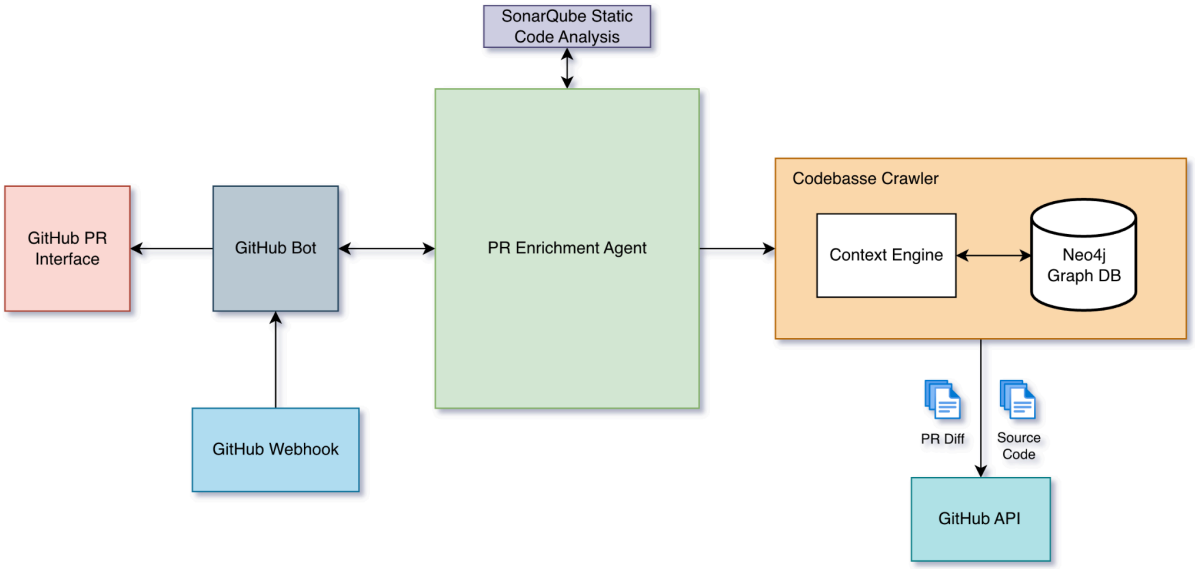

Figure 5. High Level Diagram of PR Enrichment Agent

The PR Enrichment subsystem is responsible for providing additional context and risk insights for newly opened Pull Requests (PRs). It analyzes the code changes and the

structure of the repository to highlight affected modules, identify potentially risky areas, and estimate the overall impact of the changes. This subsystem helps reviewers quickly understand which parts of the system may require closer attention during the review process.

Main Components

- GitHub Integration Layer (Webhook, Bot, & PR Interface): Acts as the communication bridge between the repository and the Mergent backend. The Webhook listens for pull request events, the GitHub Bot handles incoming event payloads and triggers the enrichment process, and the GitHub PR Interface is where the final enrichment outputs (risk summaries, impact insights) are displayed to the reviewer.
- PR Enrichment Agent: The central orchestrator of this subsystem. It manages the enrichment workflow by retrieving PR metadata and diffs, coordinating analysis components, and calculating a consolidated risk score based on multiple signals such as code changes, dependency relationships, and analysis results.
- Codebase Crawler (Context Engine & Graph DB): Responsible for understanding the structure of the repository. The Context Engine analyzes dependencies between files and modules, while the Neo4j Graph Database stores these relationships as a graph. This allows the system to identify which parts of the system may be indirectly affected by the PR changes.
- SonarQube Static Code Analysis: Provides static analysis of the modified code. It detects potential bugs, security vulnerabilities, and maintainability issues, which are used as additional signals in the PR risk evaluation.
- GitHub API: Used by the Agent and the Codebase Crawler to fetch raw data such as PR diffs, modified files, and repository source code required for the analysis.

System Workflow

1. Event Trigger: When a developer opens or updates a Pull Request, a GitHub Webhook is triggered and notifies the GitHub Bot.
2. Data Extraction: The Bot forwards the event to the PR Enrichment Agent, which retrieves the necessary PR metadata, diffs, and modified files using the GitHub API.
3. Impact & Static Analysis: The Agent sends the modified files to the Codebase Crawler to analyze dependency relationships through the graph database. At the same time, SonarQube performs static code analysis on the changes to detect potential issues.
4. Execution & Feedback: The Agent combines the impact analysis and static analysis results to calculate a risk score and identify affected modules. The results are sent back to the GitHub Bot, which posts an enrichment summary on the GitHub PR Interface to help reviewers understand the potential impact of the changes.

Functions

- **generateFileRiskAssessments:** Runs an LLM tool-calling loop to assess the risk level of each changed file. The agent may call a SonarQube tool to fetch metrics for context, then outputs a High / Medium / Low risk label with notes for each file.
- **badge:** Converts a letter grade (A through E) to a human-readable emoji badge for display in the enrichment comment.
- **bugsLetter:** Converts a raw SonarQube bug count to an A–E quality grade based on defined severity thresholds.
- **vulnsLetter:** Converts a raw SonarQube vulnerability count to an A–E security grade.
- **coverageLetter:** Converts a test coverage percentage to an A–E grade.
- **duplicationLetter:** Converts a code duplication percentage to an A–E grade.

- **complexityDensityLetter:** Converts the ratio of cyclomatic complexity to lines of code to an A–E maintainability grade.
- **prSizeLetter:** Converts the total number of added and removed lines in the PR to an A–E size grade.
- **impactSizeLetter:** Converts the percentage of repository files transitively affected by the PR's changes (from the knowledge-graph impact analysis) to an A–E grade.
- **scoreToLetter:** Averages all individual metric scores and maps the result to an overall A–E RISK grade shown in the comment header.
- **generatePREnrichmentRequest:** Bot-side HTTP client that requests the full enrichment comment from the server and returns the markdown string.
- **handleEnrichment:** Top-level GitHub event handler on the bot side. Calls generatePREnrichmentRequest and posts the resulting markdown as a comment on the Pull Request.

4.5 Vibe Reviewing Agent

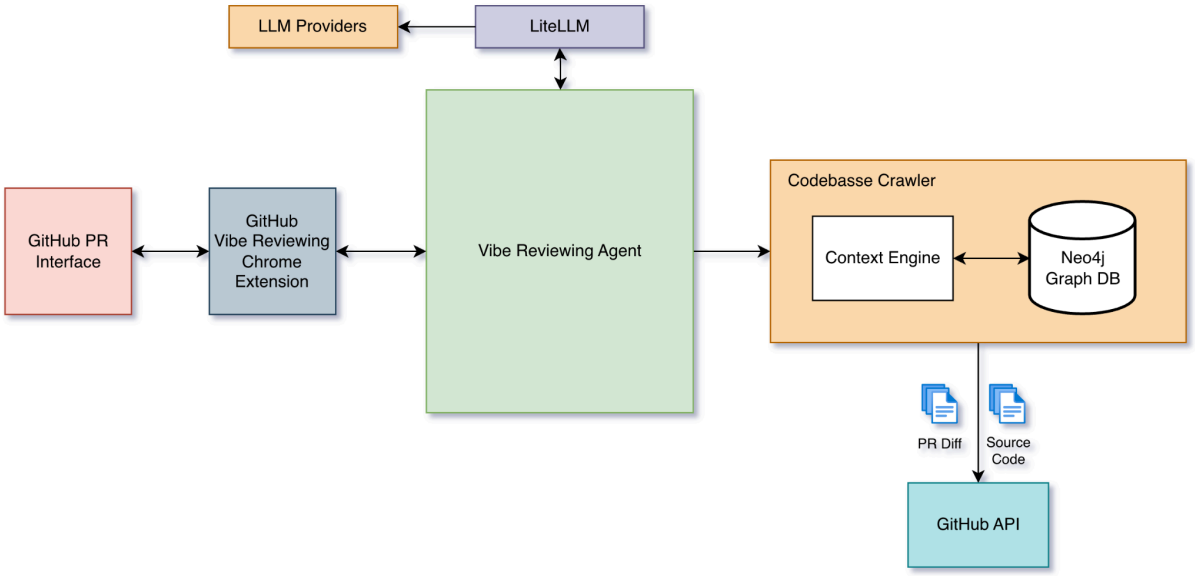

Figure 6. High Level Diagram of Vibe Reviewing Agent

The Vibe Reviewing subsystem is responsible for assisting human reviewers during the code review process by improving the clarity, tone, and usefulness of their review comments. It acts as an interactive assistant that helps reviewers better understand the code and refine their feedback before posting it. By combining repository context analysis with Large Language Model (LLM) capabilities, the subsystem enables a human–AI collaboration that improves the quality and consistency of code reviews.

Main Components

- GitHub Integration Layer (PR Interface & Chrome Extension): Acts as the interaction layer between the reviewer and the Mergent backend. The GitHub PR Interface is where the reviewer performs the code review, while the GitHub Vibe Reviewing Chrome Extension injects an interactive assistant panel into the PR page. This extension allows reviewers to ask questions about the code or request improvements to their review comments.
- Vibe Reviewing Agent: The central orchestrator of this subsystem. It manages the interaction between the reviewer, the codebase context, and the language model.

The agent processes reviewer queries, retrieves relevant code context, and generates explanations or improved review comments.

- LLM (Large Language Model) & LLM Providers: The Vibe Reviewing Agent uses a Large Language Model to generate explanations and refine reviewer comments. The LLM is accessed through external LLM Providers, which supply the underlying AI models used for natural language understanding and generation.
- Codebase Crawler (Context Engine & Neo4j Graph DB): Provides structural knowledge about the repository. The Context Engine analyzes relationships between files and modules, while the Neo4j Graph Database stores these relationships as a dependency graph. This allows the system to retrieve relevant code context when answering reviewer questions.
- GitHub API: Used by the Codebase Crawler to fetch repository data such as source code files and PR diffs required for contextual analysis.

System Workflow

1. Reviewer Interaction: While reviewing a pull request on the GitHub PR Interface, the reviewer opens the Vibe Reviewing Chrome Extension to ask questions about the code or request refinement of a review comment.
2. Request Processing: The Chrome Extension sends the request to the Vibe Reviewing Agent, which coordinates the analysis process.
3. Context Retrieval: The Agent queries the Codebase Crawler, which retrieves relevant source code and dependency information from the repository using the GitHub API and the Neo4j Graph Database.
4. LLM Analysis: The Agent sends the reviewer’s request and the retrieved code context to the LLM, which generates explanations of the code or improved versions of the review comments.
5. Execution & Feedback: The generated response is returned to the Chrome Extension and displayed within the GitHub PR Interface, allowing the reviewer to use the refined feedback directly during the review process.

Functions

- **analyzeVibeProfile:** Examines the PR diff and commit messages alongside the repository's known conventions to produce a VibeProfile capturing the inferred developer intent and the overall "mood" of the change (exploratory, defensive, refactoring, etc.).
- **detectConventionDrift:** Scans changed files for patterns that deviate from established repository conventions (naming rules, file structure, comment style) and returns a list of drift observations with file paths and suggested corrections.
- **generateVibeComment:** Synthesizes the vibe profile and convention drift findings into a single conversational PR comment. The tone adapts to the detected developer intent — encouraging for exploratory work, more direct for risky shortcuts.
- **scoreTeamAlignment:** Evaluates how well the PR aligns with the team's current sprint goals, coding standards, and open technical-debt items. Returns a 0–100 alignment score with a brief rationale string.
- **handleVibeReview:** Top-level GitHub event handler on the bot side. Orchestrates the full vibe reviewing pipeline and posts the resulting comment on the PR.

4.6 Post-Review Summary Agent

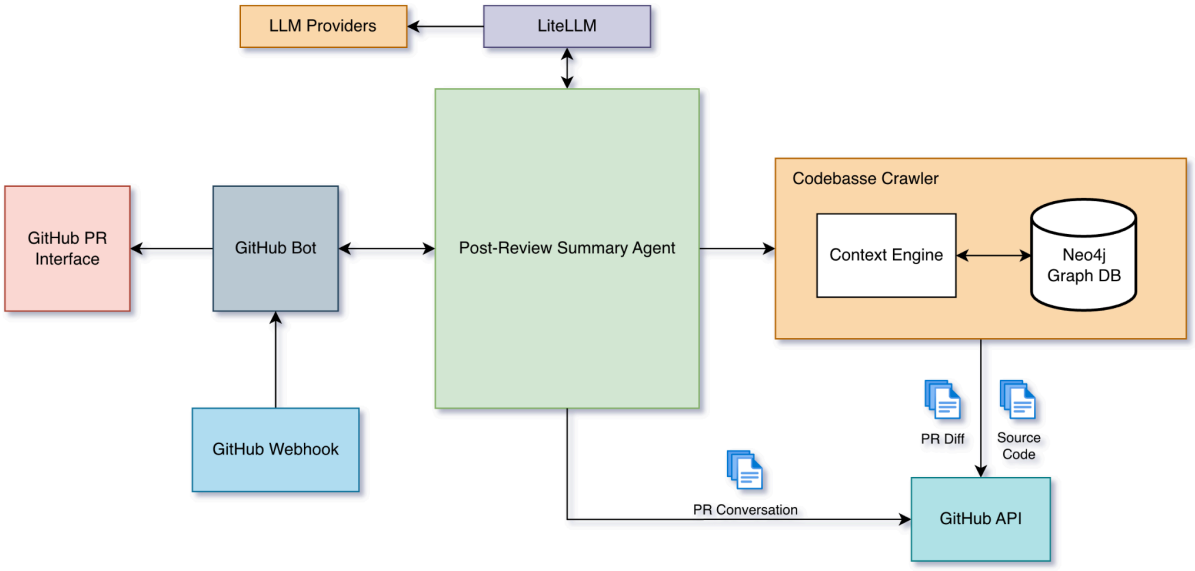

Figure 7. High Level Diagram of Post-Review Summary Agent

The Post-Review Feedback subsystem is responsible for generating a consolidated summary of the pull request after the review process is completed. It analyzes the full discussion of the PR, including AI-generated comments and human reviewer feedback, and transforms them into a clear and structured summary for the PR author. This subsystem helps developers quickly understand the main review outcomes, the key issues that were raised, and the next steps required before the PR can be merged.

Main Components

- GitHub Integration Layer (Webhook, Bot, & PR Interface): Acts as the communication bridge between the GitHub repository and the Mergent backend. The GitHub Webhook listens for PR review events, the GitHub Bot handles incoming event payloads and triggers the summary generation process, and the GitHub PR Interface is where the final post-review summary is displayed to the user.
- Post-Review Summary Agent: The central orchestrator of this subsystem. It gathers PR discussions, review comments, and metadata, and coordinates the summarization process. The agent combines these inputs and generates a structured summary that highlights key feedback points and recommended follow-up actions.
- LLM (Large Language Model) & LLM Providers: The Post-Review Summary Agent uses a Large Language Model to analyze the review discussion and generate a natural language summary. The LLM is accessed through external LLM Providers that supply the AI models used for summarization.
- Codebase Crawler (Context Engine & Neo4j Graph DB): Provides contextual information about the repository structure. The Context Engine analyzes relationships between files and modules, while the Neo4j Graph Database stores these relationships as a dependency graph. This allows the system to retrieve relevant code context when summarizing the review.
- GitHub API: Used by the Agent and the Codebase Crawler to fetch PR data such as the code diff, source code, and the full PR conversation required for the analysis.

System Workflow

1. Event Trigger: When a pull request reaches the end of the review phase or when the summary is requested, a GitHub Webhook event notifies the GitHub Bot.
2. Data Extraction: The Bot forwards the event to the Post-Review Summary Agent, which retrieves the PR conversation, review comments, code diffs, and source code using the GitHub API.
3. Context & LLM Analysis: The Agent queries the Codebase Crawler to retrieve relevant code context and then sends the collected review discussion and context to the LLM for summarization.
4. Execution & Feedback: The LLM generates a structured summary highlighting key review findings and recommended actions. The Agent sends the result back to the GitHub Bot, which posts the final post-review summary on the GitHub PR Interface.

Functions

- **aggregateReviewFeedback:** Fetches all bot-authored comments on the PR and extracts structured feedback items categorized by type (bug, style, alignment, risk).
- **prioritizeActionItems:** Ranks and deduplicates feedback items by severity and category. Returns an ordered list of action items with priority labels (Critical / High / Medium / Low) and suggested resolution steps.
- **generatePostReviewSummary:** Sends the aggregated feedback and prioritized action items to the LLM using the post_review_summary prompt. Returns a formatted markdown summary containing the overall outcome, key findings by topic, commit highlights, and the action checklist.
- **generatePostReviewRequest:** Bot-side HTTP client that requests the post-review summary from the server and returns the markdown string. Currently returns a static mock body.
- **handlePostReview:** Top-level GitHub event handler on the bot side. Listens for issue_comment.created events, guards against bot self-replies and comments on plain issues (not PRs), checks for the "/mergent post-review" trigger phrase, and posts the summary comment when triggered.

4.7 Data Dashboard

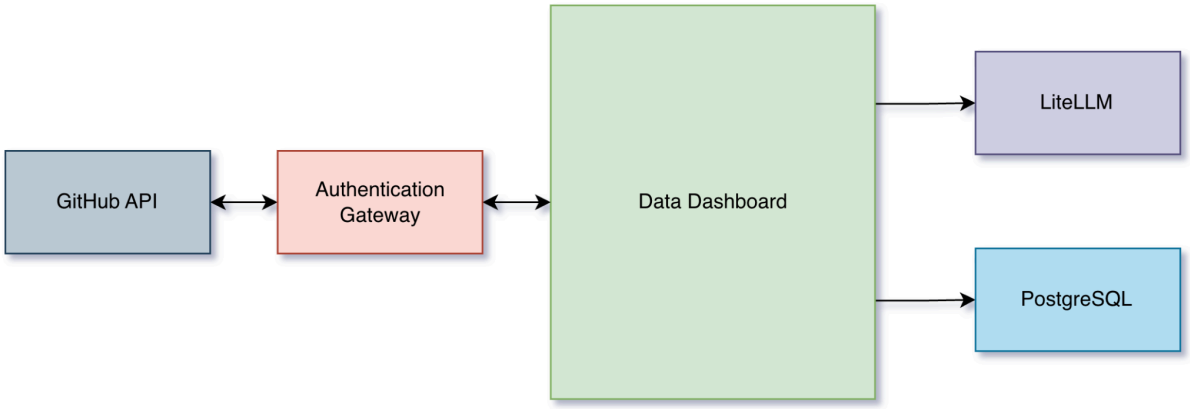

Figure 7. High Level Diagram of Data Dashboard

The Organization Monitoring Dashboard subsystem is responsible for presenting repository-level analytics and usage insights about Mergent in a single interface. Its purpose is to help team leads and developers monitor pull request activity, review efficiency, AI usage, and related costs over time.

Main Components

- GitHub API: Serves as the main external data source of the subsystem. It provides raw repository information such as pull requests, review activity, timestamps, comments, and other metadata needed for monitoring and analytics.
- Authentication Gateway: Acts as the secure communication layer between the GitHub API and the dashboard. It manages authentication and authorized access so that the dashboard can safely retrieve repository data without exposing credentials directly.
- Data Dashboard: The central component of this subsystem. It collects repository data through the Authentication Gateway, processes it, and presents analytics to the user. It is responsible for displaying metrics such as PR counts, review durations, AI usage statistics, and comparison views over time.
- LiteLLM: Supports the dashboard in AI-related monitoring tasks. It can be used to process or summarize usage-related information, especially for LLM-based operations and cost-aware analytics shown in the dashboard.
- PostgreSQL: Stores the processed dashboard data, calculated metrics, and historical records. This allows the system to keep analytics over time and present trends, comparisons, and usage statistics efficiently.

System Workflow

1. Data Retrieval: The dashboard requests repository and review data from the GitHub API through the Authentication Gateway.
2. Secure Access: The Authentication Gateway verifies access permissions and securely forwards the required data between GitHub and the dashboard.
3. Data Processing and Storage: The Data Dashboard processes the retrieved repository data, calculates meaningful analytics, and stores the results in PostgreSQL for later use and historical comparison.
4. AI Usage Monitoring: When needed, the dashboard interacts with LiteLLM to collect or summarize LLM-related usage and cost information.
5. Visualization: The dashboard presents the collected and processed information to the user as repository-level metrics, trend views, and usage summaries that help evaluate the impact of Mergent on the review workflow.

Functions

- **getRepositoryOverview:** Retrieves the stored repository record from the database and enriches it with live GitHub stats (open issues, open PRs, contributor count) for the dashboard header.
- **getIssueLinkageStats:** Computes issue linkage statistics for a repository over a date range: total issues indexed, average similarity score of linked issues, percentage of PRs with automatic linkage, and top unlinked clusters.
- **getAlignmentTrends:** Fetches historical alignment results (Exact / Missing / Tangling) from the database and groups them by week to produce a time-series dataset for trend charts.
- **getCodeQualityMetrics:** Aggregates per-PR enrichment metric scores (bugs, vulnerabilities, coverage, duplication, complexity) over a date range. Returns averages and trend direction for each metric category.
- **getTopContributors:** Ranks contributors by PR activity, average alignment score, and code review participation. Returns the top N contributors with their stats for a leaderboard section.
- **getDashboardSummary:** Composite function that calls all individual metric functions and assembles the full dashboard data payload in a single request, used by the dashboard frontend to load all data at once.

- **registerRepository:** Creates a new repository record in the database when a GitHub App installation is added. Persists the repository's GitHub ID, name, and owner for dashboard access.
- **unregisterRepository:** Removes the repository record from the database when the GitHub App installation is removed, ensuring the dashboard does not show stale data.

5 Test Cases (functional and non functional test cases, procedures and expected results)

5.1 Issue Linkage Agent Test Cases

Test ID: TC-IL-01

- **Category:** Functional / Integration
- **Severity:** Critical
- **Objective:** Verify that the system automatically detects an already linked PR and does not trigger semantic search or overwrite the link.
- **Steps:**
    1. Navigate to the test repository on GitHub.
    2. Create a new issue titled "Update user profile UI". Note the issue number (e.g., #15).
    3. Create a new branch, make a minor code change, and push it.
    4. Open a new Pull Request and include the text `Closes #15` in the PR description body.
    5. Wait for the Mergent bot to process the PR event.
- **Expected:** The Mergent bot should acknowledge the existing link and **not** post a comment suggesting other issues.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IL-02

- **Category:** Functional / Integration
- **Severity:** Critical
- **Objective:** Verify that the system suggests and automatically links the most relevant existing issue when an unlinked PR is opened.
- **Steps:**
    1. Ensure there is an open issue in the repository titled "Fix database timeout error during login".
    2. Create a new branch, add code that modifies the database connection timeout settings, and push it.
    3. Open a Pull Request with a generic title like "Update DB settings" and leave the description empty (do not link any issues).
    4. Observe the comments posted by the Mergent bot.
- **Expected:** The bot posts a comment listing the "Fix database timeout error during login" issue with a high confidence score and automatically applies the linkage (e.g., by updating the PR description with `Closes #...`).
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IL-03

- **Category:** Functional / Integration
- **Severity:** Major

- **Objective:** Verify that the system can generate and link a new issue when the PR content does not match any existing issues.
- **Steps:**
    1. Create a branch and add a completely novel feature (e.g., "Add SMS notification service"). Ensure no existing issues mention SMS notifications.
    2. Open a Pull Request for this branch without linking an issue.
    3. Observe that the bot suggests issues with very low confidence scores.
    4. Post a comment on the PR using the trigger command to generate an issue (e.g., `/mergent create issue`).
    5. Navigate to the repository's Issues tab.
- **Expected:** A new issue is automatically created containing a title and description derived from the PR's context, and the bot links this newly created issue to the PR.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IL-04

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the "Branch Match" heuristic accurately detects and rewards a PR when its branch name contains the target issue number.
- **Steps:**
    1. Navigate to the repository and create a new issue titled "Implement shopping cart logic". Note the assigned issue number (e.g., #85).
    2. Create a new branch specifically named to include the issue number, such as `feature/issue-85` or `fix-85`.
    3. Make a minor code change, commit, and push the branch.
    4. Open a Pull Request from this branch without explicitly linking the issue in the description.
    5. Wait for the Mergent bot to post the "Similar Issues Based on AI Analysis" comment.
    6. Expand the "Score Breakdown" for Issue #85 in the bot's comment.
- **Expected:** The bot lists Issue #85. In the "Score Breakdown", the "Branch Match" metric should display a positive score (e.g., `100.0% (0.30 / 0.30)`), increasing the Total Score.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IL-05

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the "Assignee Match" heuristic correctly identifies when the PR author is the same person assigned to the suggested issue.
- **Steps:**
    1. Ask Tester A to create a new issue in the repository and assign it to themselves (Tester A).
    2. Tester A creates a branch with a generic name (e.g., `update-configs`), makes a change, and pushes it.
    3. Tester A opens a Pull Request without linking the issue in the PR description.
    4. Wait for the Mergent bot to post the analysis comment.
    5. Expand the "Score Breakdown" for the issue created in Step 1.
- **Expected:** In the "Score Breakdown" for that specific issue, the "Assignee Match" metric should display a maximum score (e.g., `100.0% (0.20 / 0.20)`), whereas other suggested issues where Tester A is not assigned should show `0.0%`.

- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IL-06

- **Category:** Functional / Integration
- **Severity:** Minor
- **Objective:** Verify the UI presentation and formatting of the PR-Issue Linkage suggestion comment, ensuring exactly 5 candidates are listed with detailed breakdowns.
- **Steps:**
    - Ensure the repository has at least 6 open issues.
    - Open a new unlinked Pull Request with a generic title like "Refactor backend services".
    - Wait for the Mergent bot to post its comment.
    - Verify the title of the comment and the number of suggested items.
    - Check the bullets under any suggested issue.
- **Expected:** * The comment title is exactly "Similar Issues Based on AI Analysis".
    - Exactly 5 issues are listed and ranked (1 to 5).
    - Each issue has a "Total Score" and a "Status" badge.
    - Each issue contains a "Score Breakdown" bullet listing "Text Similarity", "Date Proximity", "Branch Match", and "Assignee Match".
- **Date-Result:** *(To be completed in Final Report)*

5.2 Issue Alignment Agent Test Cases

Test ID: TC-IA-01

- **Category:** Functional / Integration
- **Severity:** Critical
- **Objective:** Verify that the system categorizes the alignment as **Exact** when the PR fully implements the issue requirements without adding unrelated changes.
- **Steps:**
    1. Create an issue: "Change the primary navigation bar color to blue".
    2. Open a PR linked to this issue containing *only* CSS changes that alter the navigation bar color to blue.
    3. Wait for the Mergent alignment analysis to complete.
- **Expected:** The bot posts an Alignment Analysis comment categorizing the result as "Exact" and applies the corresponding "Exact" label to the PR.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IA-02

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the system categorizes the alignment as **Missing** when the PR fails to implement all required changes.
- **Steps:**
    1. Create an issue: "Implement login and password reset pages".
    2. Open a PR linked to this issue that *only* adds the login page, completely omitting the password reset page.
    3. Wait for the Mergent alignment analysis to complete.
- **Expected:** The bot posts an Alignment Analysis comment categorizing the result as "Missing", provides reasoning mentioning the absent password reset page, and applies the "Missing" label.

- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IA-03

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the system categorizes the alignment as **Tangling** when the PR includes additional, unrelated changes.
- **Steps:**
    1. Create an issue: "Fix typo in the footer text".
    2. Open a PR linked to this issue that fixes the footer typo, but *also* includes a massive refactor of the user authentication module.
    3. Wait for the Mergent alignment analysis to complete.
- **Expected:** The bot categorizes the PR as "Tangling". The reasoning comment should explicitly point out that the authentication refactoring is unrelated to the footer typo issue.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IA-04

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the system categorizes the alignment as **Missing & Tangling** when the PR fails to address requirements and adds unrelated work.
- **Steps:**
    1. Create an issue: "Create endpoints for User Profile and User Settings".
    2. Open a PR linked to this issue that creates the User Profile endpoint (missing Settings) and modifies the styling of the billing page (unrelated).
    3. Wait for the Mergent alignment analysis to complete.
- **Expected:** The bot categorizes the alignment as "Missing & Tangling". The comment should explain both the missing endpoint and the unrelated styling changes.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IA-05

- **Category:** Functional / Integration
- **Severity:** Minor
- **Objective:** Verify that the alignment agent updates its categorization when a developer pushes new commits to fix a previously "Missing" PR.
- **Steps:**
    1. Follow the steps in TC-IA-02 to get a PR labeled as "Missing" (missing the password reset page).
    2. Push a new commit to the same PR branch that successfully adds the password reset page.
- **Expected:** The bot re-evaluates the code, posts a new comment updating the status to "Exact", and replaces the "Missing" label with the "Exact" label.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IA-06

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the alignment reasoning explicitly references specific unfulfilled requirements (e.g., R8, R9) from the issue description when categorizing a PR as "Missing" or "Missing and Tangling".

- **Steps:**
    1. Create a new issue titled "Implement Vehicle Comparison Tool".
    2. In the issue description, explicitly list numbered requirements (e.g., R1: UI layout, R2: Data fetching, R3: AI integration).
    3. Create a branch and implement only R1 and R2. Purposely omit the AI integration (R3).
    4. Open a Pull Request linked to this issue.
    5. Wait for the Mergent bot to post the "Issue Alignment Analysis" comment.
    6. Read the reasoning text provided by the bot.
- **Expected:** The bot's reasoning must explicitly state which specific requirements were implemented (e.g., R1, R2) and specifically call out the omitted requirement (e.g., R3 or "AI integration") as the reason for the "Missing" categorization.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IA-07

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the alignment reasoning explicitly names the unrelated code components introduced in the PR when categorizing it as "Tangling".
- **Steps:**
    1. Create an issue titled "Update user profile layout".
    2. Create a branch and make the necessary layout changes.
    3. In the same branch, add a completely new and unrelated file/component (e.g., a file named `SocialShareModal.js` or `PaymentGateway.ts`).
    4. Open a Pull Request linked to the issue.
    5. Wait for the Mergent bot to post the "Issue Alignment Analysis" comment.
    6. Read the reasoning text.
- **Expected:** The reasoning must explicitly mention the exact name of the unrelated component (e.g., "introduces an unrelated 'SocialShareModal' component") and explain that it is outside the scope of the linked issue.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-IA-08

- **Category:** Functional / Integration
- **Severity:** Minor
- **Objective:** Verify the UI formatting of the Alignment comment and ensure that multiple appropriate GitHub labels are applied for combined categories like "Missing and Tangling".
- **Steps:**
    - Create an issue with multiple requirements.
    - Open a PR that misses some requirements AND adds an unrelated feature (to trigger the "Missing and Tangling" state).
    - Inspect the final comment posted by the `mergent-ai-dev` bot.
    - Check the GitHub labels applied to the Pull Request.
- **Expected:** * The comment title must be exactly "Issue Alignment Analysis" formatted as a Markdown header.
    - The comment must include a clear "Alignment: [Category]" conclusion line at the bottom.
    - The bot must successfully apply **two separate labels** (`missing` and `tangling`) to the GitHub PR interface.
- **Date-Result:** *(To be completed in Final Report)*

5.3 AI Code Review Agent Test Cases

Test ID: TC-RC-01

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the AI code review agent generates an inline comment with an actionable code suggestion for suboptimal code.
- **Steps:**
  - Create a repository branch and intentionally introduce a code flaw (e.g., missing a null check, using an inefficient loop).
  - Open a Pull Request containing these changes.
  - Wait for the `mergent-ai-dev` bot to complete its review process.
  - Navigate to the "Files changed" tab in the GitHub PR interface and inspect the bot's comments.
- **Expected:** * The bot must post an inline comment attached to the specific line(s) of the introduced flaw.
  - The comment must include an explanation of the issue and an actionable code fix formatted as a GitHub suggested change block (allowing the author to click "Commit suggestion").
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-RC-02

- **Category:** Non-functional / Performance
- **Severity:** Major
- **Objective:** Verify that the AI comment generation process completes within the maximum allowed time constraint of 2 minutes.
- **Steps:**
  - Open a standard-sized Pull Request (e.g., ~100-200 lines of code changed).
  - Start a timer exactly when the PR is opened (or when the review webhook is triggered).
  - Monitor the PR until the `mergent-ai-dev` bot posts its "Auto Generated AI Comments" summary and inline reviews.
  - Stop the timer.
- **Expected:** * The total elapsed time from the trigger event to the posting of the final review comments must not exceed 2 minutes.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-RC-03

- **Category:** Functional / Integration
- **Severity:** Minor
- **Objective:** Verify the AI review agent's behavior when presented with clean, well-optimized code that does not require modifications.
- **Steps:**
  - Create a PR containing simple, clean, and bug-free code additions (e.g., adding a basic markdown file or a strictly formatted interface).
  - Wait for the `mergent-ai-dev` bot to complete its review.
  - Inspect the PR conversation tab.
- **Expected:** * The bot must not hallucinate errors or post irrelevant inline code suggestions.

    - The bot should leave a general PR summary comment indicating that the code looks good overall (e.g., "Your PR looks good overall. No major issues detected.").
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-RC-04

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the AI review agent successfully analyzes and comments on multiple modified files within a single Pull Request.
- **Steps:**
    - Open a PR that includes modifications across at least 3 distinct files (e.g., a frontend component, a backend route, and a utility function).
    - Introduce a minor, detectable issue in at least two of the different files.
    - Wait for the `mergent-ai-dev` bot to complete its review.
    - Inspect the "Files changed" tab.
- **Expected:** * The bot must post inline comments on the correct lines across the different files, proving it parses and analyzes the entire diff context, not just the first file in the sequence.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-RC-05

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the AI Comment feature can be successfully retriggered and correctly evaluates updated code after new commits are added to the PR.
- **Steps:**
    - Open a PR with code flaws and wait for the initial AI review comments to be posted.
    - Push a new commit to the PR branch that resolves the suggested changes or adds new features.
    - Reactivate the AI Comment feature (e.g., via a designated PR comment command).
    - Inspect the new comments generated by the mergent-ai-dev bot.
- **Expected:** * The bot must reevaluate the newly updated code difference.
    - The bot must publish a new review reflecting the updated PR state (e.g., acknowledging the fix or reviewing the new features) without redundantly repeating comments on issues that have already been resolved.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-RC-06

- **Category:** Reliability / Non-functional
- **Severity:** Major
- **Objective:** Verify that the system handles external LLM API failures gracefully without crashing the core service or hanging indefinitely.
- **Steps:**
    - Temporarily revoke or invalidate the Azure LLM API key in the testing environment to simulate an external service outage.

    - Open a new Pull Request with modified code to trigger the review workflow.
    - Monitor the bot's behavior in the PR and check the backend service logs.
- **Expected:** * The core service must not crash or cause a system-wide outage.
    - The bot should handle the failure gracefully, ideally by logging the error and posting a fallback comment to the PR indicating that the AI review service is temporarily unavailable.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-RC-07

- **Category:** Non-functional / Scalability
- **Severity:** Minor
- **Objective:** Verify the AI review agent's behavior when processing a massive Pull Request that could potentially exceed token limits or performance constraints.
- **Steps:**
    - Open a massive Pull Request (e.g., modifying 50+ files or 5,000+ lines of code, such as an auto-generated package lock update or a massive directory refactor).
    - Trigger the AI review generation.
    - Observe the bot's output and record the processing time.
- **Expected:** * The system must not crash under the heavy payload.
    - The bot must either complete the analysis within the 2-minute constraint, safely truncate the analysis (e.g., skipping auto-generated files), or post a warning message stating the PR is too large for a full line-by-line review.
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-PRS-01

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the AI code review agent generates a correctly formatted PR Summary containing a high-level overview and a bulleted list of key changes.
- **Steps:**
    - Create a branch and implement a few distinct, identifiable changes (e.g., creating a new UI component, updating a route, and modifying a state context).
    - Open a Pull Request for these changes.
    - Wait for the `mergent-ai-dev` bot to complete its analysis and post its summary comment.
    - Inspect the formatting and structure of the posted comment.
- **Expected:** * The comment title must be exactly "PR Summary" formatted as a Markdown header.
    - The comment must include an introductory paragraph that accurately summarizes the overall purpose of the PR.
    - The comment must include a "Key Changes" section containing a bulleted list of the specific technical modifications (e.g., referencing specific components, contexts, or logic changes).
- **Date-Result:** *(To be completed in Final Report*

Test ID: TC-PRS-02

- **Category:** Functional / Accuracy
- **Severity:** High
- **Objective:** Verify that the "Key Changes" bullet points accurately reflect the code diffs without hallucinating non-existent features or missing major updates.
- **Steps:**
    - Open a Pull Request with a very specific, well-defined scope (e.g., adding a specific database migration file and updating one backend controller).
    - Wait for the `mergent-ai-dev` bot to post the PR Summary.
    - Cross-reference the bullet points in the "Key Changes" section against the actual files changed in the PR diff.
- **Expected:** * The bot must correctly identify and list the actual components/files modified.
    - The bot must **not** list any features, components, or logic changes that are not present in the PR diff (no AI hallucinations).
- **Date-Result:** *(To be completed in Final Report)*

Test ID: TC-PRS-03

- **Category:** Edge Case / Functional
- **Severity:** Minor
- **Objective:** Verify the PR Summary generation behavior when presented with a trivial or non-code Pull Request (e.g., only documentation updates or minor typo fixes).
- **Steps:**
    - Open a Pull Request that only contains minor text changes to a `README.md` file or comments, with zero functional code changes.
    - Wait for the `mergent-ai-dev` bot to post the PR Summary.
    - Inspect the generated summary and key changes.
- **Expected:** * The bot must successfully generate a "PR Summary" without failing or throwing an error due to a lack of code syntax.
    - The summary paragraph must correctly identify the PR as a documentation update.
    - The "Key Changes" section should be concise and accurately state that only text/documentation was modified.
- **Date-Result:** *(To be completed in Final Report)*

5.4 PR Enrichment Agent Test Cases

Test ID: TC-EN-01

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the system successfully calculates a consolidated risk score and displays the corresponding letter grade (A-E) in the PR comment header.
- **Steps:**
    1. Open a new Pull Request with a moderate amount of changes (e.g., 50-100 lines).

    2. Ensure SonarQube analysis and Codebase Crawler are active for the repository.
    3. Wait for the Mergent bot to post the Enrichment summary.
    4. Inspect the header of the bot's comment.
- **Expected:** The bot posts a comment containing an overall RISK grade which is an average of all individual metric scores.
- **Date-Result:** (To be completed in Final Report)

Test ID: TC-EN-02

- **Category:** Functional / Accuracy
- **Severity:** Major
- **Objective:** Verify that the PR Enrichment Agent correctly calculates and displays size-related metrics and risk grades based on the code diff.
- **Steps:**
    - Open a Pull Request containing a set of code changes (additions and deletions) across multiple files.
    - Wait for the PR Enrichment Agent to retrieve the PR metadata and unified diff via the GitHub API.
    - Allow the agent to process the diff through the prSizeLetter and impactSizeLetter functions.
    - Inspect the "Enrichment Summary" posted by the bot on the GitHub PR Interface.
- **Expected:** The bot must display an accurate count of "Number of Changed Code Files," "Total Added Lines," and "Total Deleted Lines".
    - The prSizeLetter must map the total line changes to the correct A-E size grade based on defined thresholds.
    - The scoreToLetter function must incorporate these size metrics into the overall RISK grade shown in the comment header.
    - The summary must accurately list which modules are affected by the changes as identified by the Codebase Crawler.
- **Date-Result:** (To be completed in Final Report)

Test ID: TC-EN-03

- **Category:** Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the system correctly identifies "Affected Modules" that are transitively impacted by the changes using the Neo4j dependency graph.
- **Steps:**
    1. Identify a core utility function that is imported by multiple other modules in the repository.
    2. Open a PR that modifies the signature or logic of this utility function.
    3. Observe the "Impact Insights" or "Affected Modules" section of the bot's comment.
- **Expected:** The bot must list the modules that depend on the modified file, even if those modules were not directly edited in the PR.
- **Date-Result:** (To be completed in Final Report)

Test ID: TC-EN-04

- **Category:** Functional / Integration
- **Severity:** Minor
- **Objective:** Verify the UI presentation of SonarQube metrics using the correct symbols and letters.
- **Steps:**
    1. Open a PR with specific code quality issues (e.g., duplicated code blocks or low test coverage).
    2. Wait for the enrichment comment to be posted.
    3. Check the "Static Analysis" or "Code Quality" metrics section.
- **Expected:** Each metric (Bugs, Vulnerabilities, Coverage, Duplication, Complexity) must display a letter grade (A-E) accompanied by the corresponding human-readable emoji badge.
- **Date-Result:** (To be completed in Final Report)

Test ID: TC-EN-05

- **Category:** Functional / Accuracy
- **Severity:** Major
- **Objective:** Verify that the "Impact Size" and "PR Size" grades accurately reflect the scale of the changes.
- **Steps:**
    1. Open a very large PR (e.g., adding 2,000+ lines).
    2. Open a very small PR (e.g., fixing a 1-line typo).
    3. Compare the "Size" grades for both PRs in the enrichment comments.
- **Expected:** The large PR should receive a lower size grade (e.g., 'D' or 'E'), indicating higher risk due to volume, while the small PR should receive an 'A'.
- **Date-Result:** (To be completed in Final Report)

5.5 Vibe Reviewing Agent Test Cases

Test ID: TC-VR-01

- **Category**: Functional / Integration
- **Severity:** Major
- **Objective:** Verify that the Vibe Reviewing Agent can explain a selected code change in natural language using repository context.

Steps:

1. Open a Pull Request containing several modified files.
2. Navigate to the Files changed tab in the GitHub PR interface.
3. Open the Vibe Reviewing Chrome Extension panel.
4. Select a code section and ask the assistant to explain the change.
5. Observe the response generated by the assistant.

Expected:

1. The assistant must generate a clear explanation of the selected code change.
2. The explanation must reference the code's purpose within the PR context.
3. The explanation must be displayed correctly inside the extension interface.

Date-Result: (To be completed in Final Report)

Test ID: TC-VR-02

Category: Functional / Integration

Severity: Major

Objective: Verify that the Vibe Reviewing Agent can refine a draft review comment written by the user.

Steps:

1. Open a Pull Request containing code changes.
2. Write a rough draft comment in the PR review field (e.g., "This code is confusing").
3. Open the Vibe Reviewing extension.
4. Ask the assistant to improve the review comment.
5. Inspect the generated response.

Expected:

- The assistant must generate a rewritten version of the comment.
- The rewritten comment must maintain the original meaning but improve tone and clarity.
- The comment must be constructive and suitable for posting in a code review.

Date-Result: (To be completed in Final Report)

Test ID: TC-VR-03

Category: Non-functional / Performance

Severity: Major

Objective: Verify that the Vibe Reviewing interaction completes within acceptable response time.

Steps:

1. Open a Pull Request with typical code changes.
2. Open the Vibe Reviewing extension.
3. Submit a request for code explanation or comment improvement.
4. Start timing when the request is sent.
5. Record the time until the response appears.

Expected:

- The assistant response should appear within 15 seconds.
- The interaction must remain responsive and usable within the PR interface.

Date-Result: (To be completed in Final Report)

Test ID: TC-VR-04

Category: Reliability / Integration

Severity: Major

Objective: Verify that the system handles LLM service failures gracefully.

Steps:

1. Temporarily disable or invalidate the LLM API key in the test environment.
2. Open a Pull Request and launch the Vibe Reviewing extension.
3. Submit a request for code explanation.
4. Observe the system behavior.

Expected:

- The system must not crash or freeze.
- The extension must display an informative error message indicating the AI service is unavailable.
- The backend must log the failure without interrupting the review workflow.

Date-Result: (To be completed in Final Report)

5.6 Post-Review Summary Test Cases

Test ID: TC-PRPRS-01

Category: Functional / Integration

Severity: Major

Objective: Verify that the post-review summary is generated only when the user explicitly invokes the /mergent post-review command.

Steps:

1. Open a Pull Request containing code changes across multiple files.
2. Wait for the mergent-ai-dev bot to generate AI review comments and ensure at least one human reviewer adds a manual comment.
3. Confirm that no post-review summary has been generated yet.
4. Enter the /mergent post-review command in the PR conversation.
5. Inspect the PR conversation after the command is executed.

Expected:

- No post-review summary comment must exist before the command is issued.
- After the /mergent post-review command is posted, exactly one new post-review summary comment must be created by the bot.

Date-Result: (To be completed in Final Report)

Test ID: TC-PRPRS-02

Category: Functional / Integration

Severity: Major

Objective: Verify that the post-review summary consolidates both AI-generated and human reviewer feedback when triggered by /mergent post-review.

Steps:

1. Open a Pull Request containing code changes across multiple files.
2. Wait for the mergent-ai-dev bot to generate AI review comments on the PR.
3. Ask at least one human reviewer to add a manual review comment on the same Pull Request.
4. Enter the /mergent post-review command in the PR conversation.
5. Inspect the generated post-review summary comment.

Expected:

- Exactly one summary comment must be posted after the command.
- The summary comment must contain at least one finding originating from an AI review comment.
- The summary comment must contain at least one finding originating from a human review comment already present on the PR.

Date-Result: (To be completed in Final Report)

Test ID: TC-PRPRS-03

Category: Functional / Integration

Severity: Major

Objective: Verify that the post-review summary provides explicit follow-up actions for identified review findings.

Steps:

1. Open a Pull Request with review comments requesting additional testing, refactoring, and documentation changes.
2. Ensure the comments are visible in the PR conversation or review threads.
3. Enter the /mergent post-review command in the PR conversation.
4. Inspect the generated summary comment.

Expected:

- The summary must include at least one explicit action item related to testing.
- The summary must include at least one explicit action item related to refactoring.
- The summary must include at least one explicit action item related to documentation.

Date-Result: (To be completed in Final Report)

Test ID: TC-PRPRS-04

Category: Functional / Integration

Severity: Major

Objective: Verify that the post-review summary does not contain duplicate entries when multiple comments refer to the same issue.

Steps:

1. Open a Pull Request that contains one identifiable issue in the code.
2. Add at least two similar review comments that describe the same issue, either from two human reviewers, two AI comments, or one AI and one human comment.
3. Enter the /mergent post-review command in the PR conversation.
4. Inspect the generated summary comment.

Expected:

- The summary must contain only one consolidated entry for the duplicated issue.
- The same issue must not appear multiple times as separate findings.
- The same issue must not appear multiple times as repeated action items.

Date-Result: (To be completed in Final Report)

Test ID: TC-PRPRS-05

Category: Non-functional / Performance

Severity: Major

Objective: Verify that the post-review summary is generated within an acceptable response time after the /mergent post-review command is issued.

Steps:

1. Open a Pull Request with existing AI and human review comments.
2. Start a timer immediately before posting /mergent post-review in the PR conversation.
3. Wait until the bot publishes the post-review summary comment.
4. Stop the timer.

Expected:

- Exactly one post-review summary comment must be posted after the command.
- The elapsed time between posting /mergent post-review and creation of the summary comment must not exceed the defined system threshold.

Date-Result: (To be completed in Final Report)

5.7 Data Dashboard Test Cases

Test ID: TC-DD-01

Category: Functional / Integration

Severity: Major

Objective: Verify that the dashboard displays repository productivity metrics for the selected time range.

Steps:

1. Log in as an authorized dashboard user.
2. Open the Data Dashboard.
3. Select a repository with known pull request activity.
4. Set the time range to a defined period, such as the last 6 months.
5. Inspect the productivity metrics.

Expected:

- The dashboard must display the average merge time for the selected repository and time range.
- The dashboard must display the merged pull request count for the selected repository and time range.
- Changing the time range must update the displayed values.

Date-Result: (To be completed in Final Report)

Test ID: TC-DD-02

Category: Functional / Integration

Severity: Major

Objective: Verify that the dashboard shows before-and-after comparison data relative to Mergent adoption.

Steps:

1. Prepare a repository with pull request data from both before and after the adoption date.
2. Open the Data Dashboard.
3. Navigate to the comparison section.
4. Select the test repository.

5. Inspect the comparison results.

Expected:

- The dashboard must display one metric group for the period before adoption and one for the period after adoption.
- The values in the two groups must be based on different date ranges.
- Changing the adoption date, if configurable, must refresh the comparison values.

Date-Result: (To be completed in Final Report)

Test ID: TC-DD-03

Category: Functional / Integration

Severity: Major

Objective: Verify that the dashboard lists current pull requests together with issue linkage information.

Steps:

1. Prepare a repository with open pull requests, where some are linked to issues and some are not.
2. Open the Data Dashboard.
3. Navigate to the Current Pull Requests section.
4. Inspect the listed pull requests.

Expected:

- The dashboard must display the open pull requests in the test dataset.
- Each pull request must show its linked issue when available.
- Each pull request must show a linkage status.
- Pull requests without linked issues must still be listed with an explicit unlinked or equivalent status.

Date-Result: (To be completed in Final Report)

Test ID: TC-DD-04

Category: Functional / Integration

Severity: Major

Objective: Verify that the dashboard displays timeline or calendar-based visualizations for pull requests and issues.

Steps:

1. Prepare repository data containing pull requests and issues across multiple dates.
2. Open the Data Dashboard.
3. Navigate to the timeline or calendar section.
4. Inspect the displayed items.
5. Switch views if both views are available.

Expected:

- The dashboard must display pull requests and issues according to their dates.
- Items within the selected date range must remain visible after switching between available views.
- The displayed items must correspond to the prepared dataset.

Date-Result: (To be completed in Final Report)

Test ID: TC-DD-05

Category: Functional / Integration

Severity: Major

Objective: Verify that the dashboard displays AI usage and cost analytics.

Steps:

1. Prepare a dataset containing AI-generated reviews and related analytics.
2. Open the Data Dashboard.
3. Navigate to the AI analytics section.
4. Inspect the displayed metrics.

Expected:

- The dashboard must display the number of AI-generated reviews.
- The dashboard must display the total number of AI review comments.
- The dashboard must display average review duration and average cost per review.
- The displayed values must match the prepared dataset.

Date-Result: (To be completed in Final Report)

Test ID: TC-DD-06

Category: Reliability / Security / Integration

Severity: Major

Objective: Verify that the dashboard behaves correctly under restricted access and incomplete data conditions.

Steps:

1. Attempt to open the Data Dashboard using an unauthorized account.
2. Observe the system response.
3. Then open the dashboard using an authorized account with one analytics category intentionally missing from the dataset.
4. Inspect all dashboard sections.

Expected:

- An unauthorized user must not be able to view dashboard analytics.
- The system must redirect the unauthorized user to authentication or display an access-denied message.
- When one analytics category is missing, the dashboard must still load the remaining valid sections.
- The section with missing data must display an explicit empty-state or unavailable-data message.

Date-Result: (To be completed in Final Report)

6 Consideration of Various Factors in Engineering Design

The design of Mergent is influenced primarily by economic and environmental factors due to its dependency on Large Language Models (LLMs), cloud storage, and third-party platform APIs. LLM inference introduces a direct economic cost, which affects development decisions such as model selection, caching strategies, and frequency of analysis. In addition to inference cost, storing repository data, embeddings, and metadata requires cloud storage, while running the system continuously in production requires server resources. These add to the overall operational expenses. Efficient use of cloud resources and API calls is therefore necessary to keep the system financially sustainable. Related to this, the computational needs of LLM calls and server usage introduce an environmental impact, since large-scale model usage consumes energy. This motivates design choices such as minimizing unnecessary calls, reducing storage overhead, and prioritizing efficient workflows.

Other factors, such as global and social aspects, have a minor effect on the analysis. Mergent supports collaborative software development workflows, which are used in diverse organizations, but its functionality does not depend on specific cultural practices or local norms. Likewise, the system does not influence public health or safety, nor does it introduce risks to public welfare, as it operates strictly within software development environments.

Table 1: Various Factors in Engineering Design

	Effect level	Effect

Public health	0	-
Public safety	0	-
Public welfare	0	-
Global factors	2	Used in global teams, minor influence on design.
Cultural factors	1	Review practices vary, effect is minimal.
Social factors	2	Supports collaboration but does not depend on it.
Environmental factors	4	LLM usage increases energy consumption.
Economic factors	8	LLM/API server costs shape design and frequency of use.

6.1 Constraints

Implementation Constraints

Availability of suitable training data: Limited training data for pull requests, issues, and review comments reduces the system's ability to generalize across different projects.

Reliance on LLM APIs: Mergent's performance depends on LLM quality and external API access rather than local compute power.

Integration with external platforms: GitHub or JIRA APIs have rate limits and restricted control over the review process, which limits functionality.

Limited control over workflows: Since Mergent works within existing development environments and version control systems, it can only extend workflows through APIs or bots, not modify them directly.

Economic Constraints

Cost of LLM API calls: Frequent use of LLMs for code analysis and feedback generation increases operational expenses.

Infrastructure and hosting costs: Cloud hosting, API integrations, and deployment infrastructure contribute to budget limitations.

Sustainability Constraints

High energy and resource usage: Frequent LLM calls and large-scale analyses can increase energy consumption and operational costs.

Scalability limitations: The system must remain lightweight to avoid performance degradation and excessive infrastructure demand as usage grows.

Time Constraints

Limited development time: As a two-semester capstone project, development time is finite.

Competing project activities: Research, testing, and documentation also require time, restricting the implementation of additional features and integrations beyond the core scope.

Privacy Constraints

Handling of sensitive information: Mergent processes real code, commit messages, comments, issues, and pull requests that may contain proprietary data, requiring strict confidentiality.

External API interactions: Data passed to external services must be protected to prevent leaks outside the authorized environment.

6.2 Standards

The project follows UML 2.5.1 for system and process modeling to maintain consistency and clarity in design diagrams [7]. IEEE 830-1998 standards are applied for requirements specification to ensure structured and well-documented system requirements [8]. In addition, general IEEE software engineering practices are followed to support maintainability, reliability, and quality throughout development.

7 Teamwork Details

7.1 Contributing and functioning effectively on the team

Oğuzhan Genç contributed to the architectural design and conceptual structure of the Mergent system. He participated actively in defining the system pipeline and helped shape the interaction between different components of the review workflow.

Hasan Kutluhan Şıpka mainly worked on backend infrastructure and system automation tasks. He contributed to the implementation of background services such as scheduled jobs that support the continuous operation of the system.

Kaan Aydeniz contributed to the technical infrastructure of the Mergent system. He worked on environment configuration and deployment-related tasks and helped establish the containerized development environment. In addition, he contributed to the design of automated system processes, such as scheduled background jobs used for periodically executing system tasks.

Furkan Emre Kolukırık worked on implementation tasks related to pull request analysis and risk evaluation features. His contributions focused on improving the enrichment and analysis aspects of the review pipeline.

Erdem Aydın contributed to the development of AI-assisted components of the system and participated in designing how the system integrates LLM-based analysis into the pull request review workflow.

7.2 Helping creating a collaborative and inclusive environment

Oğuzhan Genç contributed to team discussions by sharing research findings and design ideas with the team. His contributions helped the group evaluate different design alternatives and reach shared decisions.

Hasan Kutluhan Şıpka supported collaboration by assisting other team members during implementation challenges and sharing knowledge related to backend development and system infrastructure.

Kaan Aydeniz helped maintain team coordination by organizing tasks and ensuring that team members were aware of current development priorities and progress.

Furkan Emre Kolukırık contributed to a collaborative environment by discussing implementation ideas with other members and helping integrate different parts of the system.

Erdem Aydın actively participated in technical discussions and documentation activities, helping the team clarify design decisions and improve the overall quality of the project reports and system design explanations.

7.3 Taking lead role and sharing leadership on the team

Oğuzhan Genç took a leading role during the system architecture design phase by guiding discussions about the system pipeline and component interactions.

Hasan Kutluhan Şıpka led parts of the backend implementation, particularly the development of infrastructure components required for running automated system processes.

Kaan Aydeniz took initiative in organizing the project workflow through Jira and coordinating task distribution across the team.

Furkan Emre Kolukırık led the development of certain pull request analysis components, contributing to the design and implementation of risk analysis features.

Erdem Aydın took leadership in the integration of AI-based functionality into the system, helping define how AI agents interact with pull request data and contribute to the automated review process.

8 References

[1] Qodo, “Qodo: AI-Assisted Code Suggestions and PR Review Automation,” qodo.ai. Accessed: Nov. 25, 2025. [Online]. Available: https://www.qodo.ai/

[2] CodeRabbit, “AI Code Review and Automated Pull Request Comments,” coderabbit.ai. Accessed: Nov. 25, 2025. [Online]. Available: https://coderabbit.ai/

[3] LinearB, “Engineering Analytics and Workflow Automation Platform,” linearb.io. Accessed: Nov. 25, 2025. [Online]. Available: https://linearb.io/

[4] CodeScene, “Code Health and Change Impact Analysis Platform,” codescene.com. Accessed: Nov. 25, 2025. [Online]. Available: https://codescene.com/

[5] CodeAnt AI, “AI-Powered Developer Compliance and PR Analysis,” codeant.ai. Accessed: Nov. 25, 2025. [Online]. Available: https://codeant.ai/

[6] GitHub, “GitHub Copilot: AI Pair Programmer,” github.com. Accessed: Nov. 25, 2025. [Online]. Available: https://github.com/features/copilot

[7] OMG, Unified Modeling Language (UML), Version 2.5.1. Needham, MA, USA: Object Management Group, Dec. 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1

[8] "IEEE Recommended Practice for Software Requirements Specifications," in IEEE Std 830-1998 , vol., no., pp.1-40, 20 Oct. 1998, doi: 10.1109/IEEESTD.1998.88286.